

Green's functions via iterated Equations of Motion (iEoM)

User Guide

Joshua Althüser

2026

1 Prerequisites

Valid installations of

- C++ 20 and a functioning compiler (tested with g++ 13.3.0 on WSL and g++ 11.5.0 on Red Hat)
- (recommended) CMake 3.30 or newer (tested with version 3.31.8)
 - <https://cmake.org/>
 - used for easy installation running of tests
- Eigen (tested with version 3.4.1)
 - <https://gitlab.com/libeigen/eigen>
- (recommended) OpenMP
 - <https://www.openmp.org/>
 - used for parallelization of the algorithms. If it is not desired, certain preprocessor flags must be set, cf. Section 3.
- (recommended) nlohmann/json (tested with version 3.11.3)
 - <https://json.nlohmann.me/>
 - <https://github.com/nlohmann/json>
 - The output of the algorithms can be converted into `nlohmann::json` objects. If `nlohmann/json` is not available, a preprocessor flag must be set, cf. Section 3.
- (optional) BLAS and/or LAPACK: Eigen can delegate certain tasks (such as matrix diagonalization) to those libraries, which can be parallelized and be faster. Read the Eigen documentation for details.

2 Overview

The iEoM library provides an implementation of the algorithms presented in Ref.¹ and in Ref.².

The first algorithm¹ efficiently computes Fourier-transformed Green's functions. It requires an initial set $\mathfrak{S}$ of linearly independent operators. Then, one defines the dynamical matrix

$$\mathcal{M}_{ij} := \langle [O_i^\dagger, [\mathcal{H}, O_j]] \rangle, \quad (1)$$

and the norm matrix

$$\mathcal{N}_{ij} := \langle [O_i^\dagger, O_j] \rangle. \quad (2)$$

¹J. Althüser and G. S. Uhrig, Phys. Rev. B **109**, 205153 (2024), <https://doi.org/10.1103/PhysRevB.109.205153>

²J. Althüser and G. S. Uhrig, arXiv:2605.20059 (2026), <https://doi.org/10.48550/arXiv.2605.20059>

Here, O_i are the operators contained in $\mathfrak{S}$. One may always recombine the operators into Hermitian and anti-Hermitian operators,

$$\mathfrak{S}_{XP} = \{X_i := O_i + O_i^\dagger \mid O_i \in \mathfrak{S}\} \cup \{P_i := O_i - O_i^\dagger \mid O_i \in \mathfrak{S}\}, \quad (3)$$

where redundant operators are omitted such that the resulting set remains linearly independent while spanning the same operator space.

The matrices $\mathcal{M}$ and $\mathcal{N}$ now decompose into a block structure. In particular, if all relevant expectation values are real, large blocks of zeros appear:

$$\mathcal{M} = \begin{pmatrix} \mathcal{K}_+ & \underline{0} \\ \underline{0} & \mathcal{K}_- \end{pmatrix} \quad \mathcal{N} = \begin{pmatrix} \underline{0} & \mathcal{L} \\ \mathcal{L}^\dagger & \underline{0} \end{pmatrix} \quad (4)$$

The diagonal blocks correspond to the Hermitian (X) and anti-Hermitian (P) type operators, respectively. The off-diagonal blocks mix them. This block structure is leveraged by `XPResolvent` to speed up the calculations. If the expectation values are not real, or, for whatever reason, one does not want to order the operator set in the way described above, the implementation in `GeneralResolvent` can be used. However, it must be noted that the algorithms in `XPResolvent` are much more fleshed out because they could always be used in the systems I studied.

The implementations in both classes are called via `compute_collective_modes(int n)`. This code performs the required manipulations of the matrices described in Ref.¹ and, consequently, applies the Lanczos algorithm to tridiagonalize the final matrix; `n` Lanczos iterations are performed. The result is stored in a vector of `ResolventDataWrapper` objects. Each element corresponds to one starting state previously set via `create_starting_states()`. The structure of the `ResolventDataWrapper` will be explained later.

The second algorithm² extends the first one. It can additionally compute the linear combinations of operators that excite any given energy level or mode. These operators are coined *eigenoperators* and have the property

$$\mathcal{E}_n = \omega_n^2 [\mathcal{H}, [\mathcal{H}, \mathcal{E}_n]], \quad (5)$$

with the mode energy ω_n . These operators can be represented using the operator basis $\mathfrak{S}$ via

$$\mathcal{E}_n = \sum_i \psi_i^{(n)} O_i. \quad (6)$$

The coefficients $\psi_i^{(n)}$ are called *operator amplitudes*. This algorithm computes these operator amplitudes. It has been implemented only for the special case mentioned above, i.e., in `XPResolvent`. It should be possible to extend it to the general case as well, but it requires additional careful thought.

In the following, the classes will be described. Then, we discuss the example in `tests/ieom_bcs.cpp`, which computes the Green's functions for a simple BCS-type problem. Everything user-facing is defined in the namespace `mrock::iEoM`. The namespace `mrock::iEoM::detail` contains various functions that are used under the hood.

Generally, both `XPResolvent` and `GeneralResolvent` require the user to derive from them. Moreover, the user must define and override the virtual functions `create_starting_states()`, `fill_M()`, and `fill_matrices()`:

```
1 struct MyResolvent : public XPResolvent<double> {
2     void fill_M() override { /* ... */ }
3     void fill_matrices() override { /* ... */ }
4     void create_starting_states() override { /* ... */ }
5
6     MyResolvent() : XPResolvent<double>{/* ... */} { /* ... */ }
7 };
```

3 Preprocessor flags and parallelization

By default, certain sections of the code are parallelized using OpenMP. If this is not desired, one needs to define the preprocessor flag `MROCK_IEOM_DO_NOT_PARALLELIZE`.

Additionally, the algorithm can identify if the matrices have a block diagonal structure, e.g.,

$$\begin{pmatrix} 1 & 0 & 2 & 0 \\ 0 & 3 & 0 & 6 \\ 2 & 0 & 4 & 0 \\ 0 & 6 & 0 & 1 \end{pmatrix} \text{ can be transformed to } \begin{pmatrix} 1 & 2 & 0 & 0 \\ 2 & 4 & 0 & 0 \\ 0 & 0 & 3 & 6 \\ 0 & 0 & 6 & 1 \end{pmatrix}. \quad (7)$$

Then one can perform matrix operations on the blocks individually, which significantly speeds up most calculations. `GeneralResolvent` always tries to identify such blocks. For `XPResolvent`, this is optional and can be toggled via the `bool pivot` flag when calling the constructor.

These matrix operations can be performed in parallel. By default, this is *disabled*. If you want to enable it, define `MROCK_IEOM_PARALLELIZE_BLOCKMATRIX`. The relevant sections are parallelized using OpenMP.

The algorithm's output can be converted to `nlohmann::json` objects. The functionality to do so is defined via the `to_json()` functions in `ResolventDataTypes.hpp`. Naturally, an existing installation of the `nlohmann/json.hpp` is required. If this is not possible, the preprocessor flag `MROCK_IEOM_NO_NLOHMANN_JSON` must be set.

4 Overview of the classes

4.1 XPResolvent

`XPResolvent` implements the algorithm based on the *XP*-structured set (3). It is a class template with the parameters

- **RealType**: the floating-point scalar type (for example `double`),
- **n_residuals**: the number of residual eigenvectors to retain in the diagonalization-based variants²,
- **check_qr**: Computing the operator amplitudes involves a QR decomposition. This switch toggles whether the result of this decomposition should be checked. The check verifies that the product of the resulting vector with the dynamical and transformation matrices equals the target eigenvalue times the exact eigenvector. See Ref.² below Eq. (31) for details.

The class stores the two diagonal blocks of the dynamical matrix in `K_plus` ($\mathcal{K}_+$) and `K_minus` ($\mathcal{K}_-$), and the block of the norm matrix in `L` ($\mathcal{L}$), see Eq. (4). As mentioned above, the user must derive from this class and implement the following three functions

- **fill_M()**: fill the blocks `K_plus` and `K_minus`,
- **fill_matrices()**: fill both the dynamical blocks and the norm block `L`,
- **create_starting_states()**: define the initial vectors used by the Lanczos algorithm, and save them in `starting_states`.

The most important public methods are

- **compute_collective_modes()**: run the Lanczos-based collective-mode computation,
- **compute_collective_modes_with_residuals()**: compute the same result while retaining residual eigenvector information. This function works great if one only cares about eigenvectors corresponding to well-separated eigenvalues,

- `full_diagonalization()`: performs a full diagonalization of the matrix and thereby gains access to eigenvectors of bunched-up eigenvalues,
- `dynamic_matrix_is_negative()`: check whether the dynamical matrix has negative eigenvalues.

The first three functions have the template parameter `int CheckHermitian`, which defaults to -1 . If this value is > 0 , the algorithms compute $\|\mathcal{K}_\pm - \mathcal{K}_\pm^\dagger\|$ and check whether the result is smaller than 10^{-x} with $x = \text{CheckHermitian}$.

Additionally, the class provides matrix accessors for the arguments `row` and `col`, which specify the positions of elements in the original matrices $\mathcal{M}$ and $\mathcal{L}$.

- `M(row, col)`: points to the corresponding element in either `K_plus` or `K_minus`,
- `N(row, col)`: points to the corresponding element `L`.

The user-facing attributes of the class are

- `Eigen::Matrix<RealType, Eigen::Dynamic, Eigen::Dynamic> K_plus, K_minus, L,`
- `std::vector<XPStartingState<RealType>> starting_states.`

4.2 XPStartingState

`XPStartingState` is a small wrapper around two `Eigen::Vector` objects, one for the phase-like part (anti-Hermitian operators) and one for the amplitude-like part (Hermitian operators). It is used to specify the initial states for the Lanczos procedure. Its attributes are

- `phase_state`: the vector for the phase-like sector,
- `amplitude_state`: the vector for the amplitude-like sector,
- `name`: an optional identifier used in the output data.

For example, if you are interested in the phase and amplitude channel for a certain Green's function — which we will call `A` — you can do the following

```
1 StartingState A_state{ Vector::Zero(/* number of phase-like operators */), Vector::
Zero(/* number of amplitude-like operators */), "A" };
2 // for (int i=0; ...)
3 A_state.amplitude_state(i) = value;
4 A_state.phase_state(i) = value;
```

Conversely, if you only need the amplitude channel, you may use

```
1 StartingState B_state{ Vector{ }, Vector::Zero(/* number of amplitude-like operators
*/), "B" };
2 // for (int i=0; ...)
3 B_state.amplitude_state(i) = value;
4 // The following is equivalent
5 StartingState C_state = XPResolvent<...>::OnlyAmplitude(/* number of amplitude-like
operators */ , "C");
6 C_state.amplitude_state(i) = value;
7 // or for phase states
8 StartingState D_state = XPResolvent<...>::OnlyPhase(/* number of phase-like
operators */ , "D");
```

The helper functions `phase_size()`, `amplitude_size()`, and `total_size()` count how many such states are present.

4.3 GeneralResolvent

GeneralResolvent provides a more general algorithm. It is useful whenever the problem does not decompose into the block structure of Eq. (4). This class is less fleshed out than **XPResolvent**. It implements only the algorithm of Ref.¹ in `compute_collective_modes()` and the check whether $\mathcal{M}$ is positive semi-definite in `dynamic_matrix_is_negative()`.

It is a class template with a parameter `NumberType` that must be a scalar type (for example, `double` or `std::complex<double>`). The class stores the dynamical matrix in $\mathbf{M}$ ($\mathcal{M}$) and the norm matrix in $\mathbf{N}$ ($\mathcal{N}$). As mentioned above, the user must derive from this class and implement the following three functions

- `fill_M()`: fill $\mathbf{M}$,
- `fill_matrices()`: fill both $\mathbf{M}$ and $\mathbf{N}$,
- `create_starting_states()`: define the initial vectors used by the Lanczos algorithm, and save them in `starting_states`. If the starting states should be labeled, the corresponding names should be provided in `resolvent_names`.

The user-facing attributes of the class are

- `Eigen::Matrix<NumberType, Eigen::Dynamic, Eigen::Dynamic> M, N,`
- `std::vector<Eigen::Vector<NumberType, Eigen::Dynamic>> starting_states,`
- `std::vector<std::string> resolvent_names.`

4.4 Resolvent

Resolvent implements the Lanczos algorithm. It stores a starting state, runs the Lanczos recurrence, and computes the coefficients that are later interpreted as resolvent data. The class supports several overloads of `compute()`, depending on whether the problem is Hermitian, symplectic, or expressed through an explicit $\mathbf{N}$ matrix. These are used internally by both **GeneralResolvent** and **XPResolvent**, but can also be used to apply the Lanczos algorithm to other problems.

4.5 Data containers

These classes are simply containers for the data computed via the algorithms described above. All of them have the template parameter `RealType`, which expects a scalar type (for example `double`).

4.5.1 ResolventData

ResolventData is a simple class template that stores one set of Lanczos coefficients as two `std::vector` objects, `a_i` and `b_i`.

4.5.2 ResolventDataWrapper

A **ResolventDataWrapper** groups several **ResolventData** objects under a common name and is the form returned by most functions of **GeneralResolvent** and **XPResolvent**.

4.5.3 ResidualInformation

Additionally, it has the template parameter `int n`, which specifies the number of eigenvectors to compute. It contains various `std::array` objects of size `n`:

- `eigenvalues`: the lowest `n` eigenvalues,

- **eigenvectors**: the corresponding eigenvectors
- **weights**: the weights of the corresponding mode,
- **residuals**: the residual of the eigenvector; it gives information about how well the corresponding eigenvector has converged. A low value is good.
- **n_ghosts**: the number of Lanczos ghosts (certain eigenvalues may appear multiple times if too many iterations are performed).
- **converged**: Whether or not the corresponding eigenvector has converged.

4.5.4 FullDiagonalizationData

Additionally, it has the template parameter `int n`, which specifies how many eigenvectors to store. It contains three attributes

- `std::vector<RealType> eigenvalues`: all eigenvalues of the matrix that was solved
- `std::array<std::vector<RealType>, n> first_eigenvectors`: the eigenvectors of the lowest `n` eigenvalues
- `std::list<std::vector<RealType>> weights`: the inner vectors contain the weights of each eigenvalue. The outer list corresponds to different starting states.

5 Example

The code of this example is located in `tests/ieom_bcs.cpp`. We consider the Hubbard model in momentum space

$$\mathcal{H} = \sum_{k,\sigma} \varepsilon_k c_{k\sigma}^\dagger c_{k\sigma} + \frac{U}{N} \sum_{kk'q} c_{k\uparrow}^\dagger c_{k'\downarrow}^\dagger c_{k'-q\downarrow} c_{k+q\uparrow}, \quad (8)$$

where $c_{k\sigma}$ annihilates an electron with momentum k and spin σ . We consider a 1D chain with nearest-neighbor hopping and N_k equidistant momenta. The dispersion thus reads

$$\varepsilon_k = -2 \cos(k). \quad (9)$$

Moreover, we focus on $U < 0$, so the system exhibits a $SU(2)$ pseudospin symmetry that manifests in a coexistence of a charge-density wave and s -wave superconductivity. We use the aforementioned symmetry to eliminate the charge-density wave and focus solely on superconductivity. Then, the considered operator set is

$$\mathfrak{S} = \{c_{-k\downarrow} c_{k\uparrow} + \text{H.c.} \mid k \in 1^{\text{st}} \text{ BZ}\} \quad (10a)$$

$$\cup \{c_{k\uparrow}^\dagger c_{k\uparrow} + c_{-k\downarrow}^\dagger c_{-k\downarrow} \mid k \in 1^{\text{st}} \text{ BZ}\} \quad (10b)$$

$$\cup \{c_{-k\downarrow} c_{k\uparrow} - \text{H.c.} \mid k \in 1^{\text{st}} \text{ BZ}\}. \quad (10c)$$

The necessary expectation values are computed with respect to the mean-field ground state.

5.1 Evaluating spectral functions

First, we derive from `XPresolvent` and define the necessary methods

```

1 struct BCSTester : public XPresolvent<double /* ... */> {
2     void fill_M() override { /* ... */ }
3
4     void fill_matrices() override { /* ... */ }
5
6     void create_starting_states() override { /* ... */ }
7
8     BCSTester( /* ... */ )
9         : XPresolvent<double /* ... */>(5e-7, // target precision
10            2*N_k, // 2N_k amplitude operators
11            N_k, // N_k phase operators
12            false, // no need to pivot
13            true), // It is an error if M is negative
14            /* ... */
15        { /* ... */ }
16 };

```

The $5e-7$ means that the target precision of the algorithm is 5×10^{-7} . Note that the algorithm involves square roots. Thus, a precision higher than the square root of machine precision is meaningless.

There are $2N_k$ Hermitian and N_k anti-Hermitian operators. Thus, filling the matrices is done via

```

1 void fill_M() override {
2     K_plus = Matrix::Zero(2*N_k, 2*N_k);
3     K_minus = Matrix::Zero(N_k, N_k);
4     for (int k=0; k<N_k; ++k) {
5         // K-diagonal filling
6         /* ... */
7         for (int l=0; l<N_k; ++l) {
8             // K-off-diagonal filling
9             /* ... */
10        }
11    }
12 }
13 void fill_matrices() override {
14     fill_M();
15     L = Matrix::Zero(2 * N_k, N_k);
16     for (int k=0; k<N_k; ++k) {
17         // Only the k-diagonal components of L are non-zero in this setup.
18         /* ... */
19     }
20 }

```

We care about two Green's functions: the amplitude (or Higgs) channel

$$G_{\text{Higgs}} \sim \sum_{k,k'} \langle (c_{-k\downarrow} c_{k\uparrow} + \text{H.c.}) (c_{-k'\downarrow} c_{k'\uparrow} + \text{H.c.}) \rangle \quad (11)$$

and the phase channel

$$G_{\text{Phase}} \sim \sum_{k,k'} \langle (c_{-k\downarrow} c_{k\uparrow} - \text{H.c.}) (c_{-k'\downarrow} c_{k'\uparrow} - \text{H.c.}) \rangle. \quad (12)$$

These correspond to the following starting states

```

1 void create_starting_states() override {
2     starting_states.clear();
3     // 'Vector' is an Eigen::VectorXd defined in the base class.
4     starting_states.push_back(XPStartingState < double >{
5         Vector::Ones(N_k), // Phase state
6         Vector::Zero(2*N_k), // Amplitude state

```

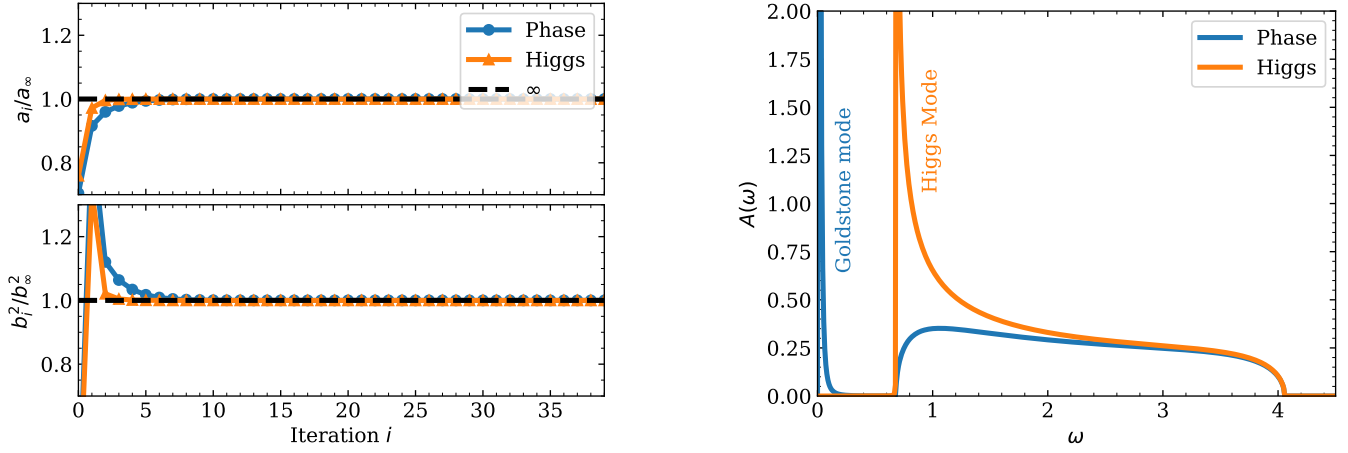


Figure 1: Left: Lanczos coefficients evaluated by `tester/ieom_bcs.cpp`. Right: The spectral function evaluated from these coefficients.

```

7     "SC" // Name
8 });
9 for (int i=0; i < N_k; ++i) {
10     // Set amplitude-state pattern (all pair creation operators)
11     starting_states.front().amplitude_state(i) = 1.;
12 }
13 starting_states.front().amplitude_state.normalize();
14 starting_states.front().phase_state.normalize();
15 }

```

The main method first solves the self-consistency problem to find the order parameter Δ . Then, we create an object of our resolvent class and can call `compute_collective_modes()`

```

1 BCSTester tester(delta_new);
2 std::vector<BCSTester::ResolventReturnData> result
3     = tester.compute_collective_modes<11>(N_lanczos);

```

Here, 11 is the value of `checkHermitian` (cf. Section 4.1), and `N_lanczos` tells the algorithm the number of Lanczos iterations to carry out. Now we can save the result to a file and use it later. One example usage is provided in `tests/bcs_spectral.py`. This script evaluates the continued fraction representing the Green's functions truncated via a square-root terminator; see Ref.¹. It creates the plots shown in Fig. 1.

5.2 Evaluating operator amplitudes

Only excitations whose energies lie outside of continua can be directly associated with a single energy eigenvalue. Here, the only such excitation is the Goldstone mode in the phase channel. We will compute its operator amplitudes, i.e., the coefficients ψ_k such that

$$\mathcal{E} = \sum_k \psi_k (c_{-k\downarrow} c_{k\uparrow} - \text{H.c.}) \quad (13)$$

fulfills Eq. (5) (on the level of the applied approximations).

Since we care only about the phase channel, we extend our original tester class

```

1 struct BCSTester : public XPResolvent<double, 1, true>
2 { /* ... */ };
3 struct PhaseTester : public BCSTester {
4     void create_starting_states() override {
5         starting_states.clear();

```

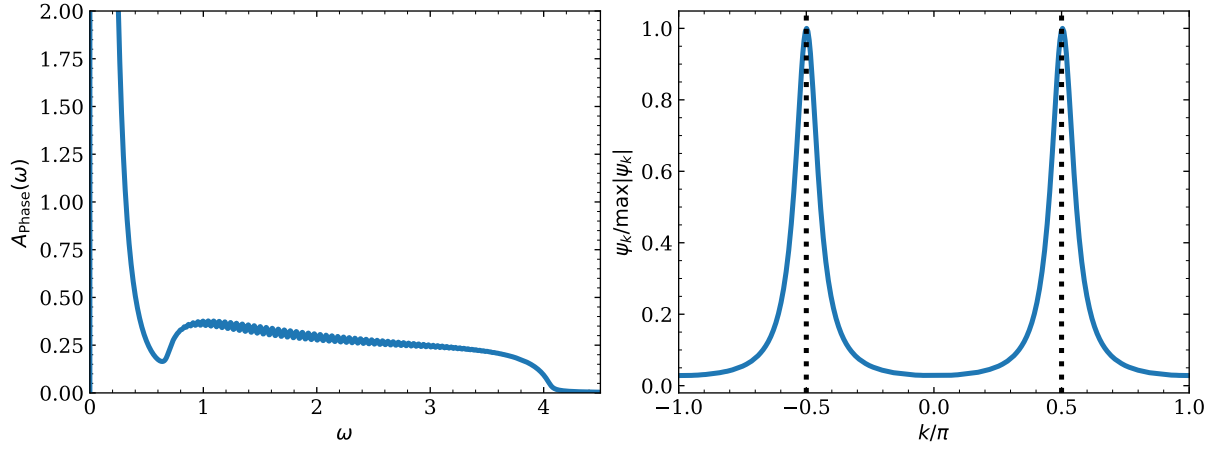


Figure 2: Left: The spectral function evaluated from the entire spectrum of the system evaluated in `tester/ieom_bcs.cpp`. The frequencies are shifted into the complex plane to visualize the occurring δ -peaks; $\omega \rightarrow \omega + 3i \cdot 10^{-2}$. Right: The operator amplitudes of the Goldstone mode. The dotted lines mark the momenta k with $\varepsilon_k = 0$.

```

6     starting_states.push_back(OnlyPhase(N_k, "SC"));
7     starting_states.front().phase_state.fill(1.);
8     starting_states.front().phase_state.normalize();
9 }
10 PhaseTester(/* ... */) : BCSTester(/* ... */) {}
11 };

```

The template parameters of `XPResolvent` mean

- **double**: The class works with **double** as the floating-point type,
- **1**: We retain only one eigenvector; the one corresponding to the lowest energy eigenvalue,
- **true**: We check the result of the QR decomposition and print it to the console.

Now, one can create an object of this class and call the corresponding function

```

1 PhaseTester phase_tester(delta_new);
2 // Both objects are of the type FullDiagonalizationData<double, 1>
3 // amplitude_data is empty because we did not specify an amplitude starting state
4 const auto [phase_data, amplitude_data] = phase_tester.full_diagonalization();

```

The result is saved and plotted in Fig. 2. The left panel shows the spectral function $A_{\text{Phase}}(\omega)$ evaluated over the full system spectrum, along with the weights corresponding to the starting state. Note that the shift of the frequencies into the complex plane is larger than before to mitigate the effects of finite discretization inside the continuum. The right panel shows the operator amplitudes ψ_k .